

**Московский авиационный институт
(национальный исследовательский университет)**

**Факультет информационных технологий и прикладной
математики**

Кафедра вычислительной математики и программирования

Лабораторная работа №4 по курсу «Дискретный анализ»

Студент: М. А. Хорохорин
Преподаватель: А. А. Кухтичев
Группа: М8О-206БВ-24
Дата: 11.05.2026
Оценка:
Подпись:

Москва, 2026

Лабораторная работа №4

Задача: Необходимо реализовать один из стандартных алгоритмов поиска образцов для указанного алфавита.

Вариант алгоритма: Поиск одного образца при помощи алгоритма Кнута—Морриса—Пратта.

Вариант алфавита: Числа в диапазоне от 0 до $2^{32} - 1$.

Запрещается реализовывать алгоритмы на алфавитах меньшей размерности, чем указано в задании.

Образец задаётся на первой строке входного файла. Затем следует текст, состоящий из чисел, в котором нужно найти заданный образец. Для каждого вхождения выводятся два числа через запятую: номер строки и номер слова в строке, с которого начинается найденный образец. Нумерация начинается с единицы.

1 Описание

Требуется реализовать алгоритм Кнута—Морриса—Пратта для поиска одного образца в тексте над алфавитом чисел от 0 до $2^{32} - 1$.

Алгоритм КМП работает за $O(|P| + |T|)$, где $|P|$ — длина образца, а $|T|$ — длина текста. Сначала для образца строится префикс-функция π : значение $\pi[i]$ равно длине наибольшего собственного префикса $P[0..i]$, который одновременно является суффиксом этой последовательности. Затем текст просматривается слева направо. При несовпадении алгоритм не возвращается назад по тексту, а использует π для выбора следующей длины уже совпавшего префикса.

Числа считываются в тип `uint32_t`. Стандартное числовое считывание корректно обрабатывает ведущие нули: `0011` и `11` становятся одним и тем же значением `11`.

2 Исходный код

1 Префикс-функция

Функция `BuildPrefixFunction` строит массив π для образца. Переменная `k` хранит текущую длину совпавшего префикса. Пока следующий символ не подходит, значение `k` откатывается по уже посчитанным значениям префикс-функции.

Листинг 1: Построение префикс-функции

```
1 | std::vector<int> BuildPrefixFunction(const std::vector<TToken> &pattern) {
2 |     int patternLen = static_cast<int>(pattern.size());
3 |     std::vector<int> prefixFunc(patternLen, 0);
4 |
5 |     for (int i = 1; i < patternLen; ++i) {
6 |         int k = prefixFunc[i - 1];
7 |         while (k > 0 && pattern[i] != pattern[k]) {
8 |             k = prefixFunc[k - 1];
9 |         }
10 |         if (pattern[i] == pattern[k]) {
11 |             ++k;
12 |         }
13 |         prefixFunc[i] = k;
14 |     }
15 |
16 |     return prefixFunc;
17 | }
```

2 Поиск

В основном решении текст обрабатывается потоково, построчно и пословно. Это важно, поскольку в условии нет ограничений на длину текста, а лимит памяти составляет 64 МБ. Для вывода позиции найденного вхождения хранится только очередь последних $|P|$ позиций `positionWindow`. Сам алгоритм остаётся классическим КМП: переменная `matched` хранит длину совпавшего префикса образца.

Листинг 2: Поточковый проход КМП в `main`

```
1 | while (ss >> token) {
2 |     positionWindow.emplace_back(lineNumber, wordNumber);
3 |     if (static_cast<int>(positionWindow.size()) > patternLen) {
4 |         positionWindow.pop_front();
5 |     }
6 |
7 |     while (matched > 0 && token != pattern[matched]) {
```

```

8      matched = prefixFunc[matched - 1];
9  }
10  if (token == pattern[matched]) {
11      ++matched;
12  }
13  if (matched == patternLen) {
14      const TPosition &match = positionWindow.front();
15      std::cout << match.first << ", " << match.second << "\n";
16      matched = prefixFunc[matched - 1];
17  }
18
19      ++wordNumber;
20 }

```

Для теста производительности также реализована функция `KmpSearch`, принимающая готовые векторы текста и позиций. Она использует тот же цикл КМП и нужна только для удобного сравнения с наивным алгоритмом в бенчмарке.

3 Консоль

```
# cmake -B build -DCMAKE_BUILD_TYPE=Release
# cmake --build build
# ./build/lab4/lab4_main
11 45 11 45 90
0011 45 011 0045 11 45 90    11
45 11 45 90
1,3
1,8
```

4 Тест производительности

1 Методика

Тест сравнивает реализованный алгоритм Кнута—Морриса—Пратта с наивным поиском. Наивный алгоритм для каждой позиции текста последовательно проверяет совпадение с образцом, поэтому в худшем случае работает за $O(|P| \cdot |T|)$. КМП строит префикс-функцию и затем просматривает текст один раз, что даёт $O(|P| + |T|)$.

Проводятся две серии экспериментов. В первой серии генерируется случайный текст над алфавитом из 10 случайных чисел, длина образца равна 8. Во второй серии используется худший случай для наивного поиска: образец состоит из 19 единиц и завершающей двойки, а текст состоит только из единиц.

Время поиска замеряется с помощью `std::chrono::high_resolution_clock`. Генерация данных и чтение входа в замер не входят. После каждого запуска бенчмарк сравнивает полные списки найденных позиций для КМП и наивного алгоритма.

2 Результаты

$ T $	КМП, мкс	Наивный поиск, мкс
10 000	27	23
100 000	314	293
500 000	951	959
1 000 000	1 813	1 731
5 000 000	10 324	9 712

Таблица 1: Случайный текст, алфавит из 10 чисел, длина образца 8

$ T $	КМП, мкс	Наивный поиск, мкс
100 000	1 610	2 310
500 000	1 186	5 644
1 000 000	2 708	10 895
5 000 000	11 924	55 733

Таблица 2: Худший случай для наивного поиска

```
# bash lab4/benchmark/run_bench.sh
=== Random text,alphabet=10,pattern_len=8 ===
n=10000:
KMP time: 27us
```

```

Naive search time: 23us
KMP matches: 0
Naive matches: 0
Results equal: yes
n=100000:
KMP time: 314us
Naive search time: 293us
KMP matches: 0
Naive matches: 0
Results equal: yes
n=500000:
KMP time: 951us
Naive search time: 959us
KMP matches: 0
Naive matches: 0
Results equal: yes
n=1000000:
KMP time: 1813us
Naive search time: 1731us
KMP matches: 0
Naive matches: 0
Results equal: yes
n=5000000:
KMP time: 10324us
Naive search time: 9712us
KMP matches: 0
Naive matches: 0
Results equal: yes

```

=== Worst case for naive: pattern=AAA...AB,text=AAA...A ===

```

n=100000:
KMP time: 1610us
Naive search time: 2310us
KMP matches: 0
Naive matches: 0
Results equal: yes
n=500000:
KMP time: 1186us
Naive search time: 5644us
KMP matches: 0
Naive matches: 0

```

```
Results equal: yes
n=1000000:
KMP time: 2708us
Naive search time: 10895us
KMP matches: 0
Naive matches: 0
Results equal: yes
n=5000000:
KMP time: 11924us
Naive search time: 55733us
KMP matches: 0
Naive matches: 0
Results equal: yes
```

3 Анализ результатов

На случайных данных наивный поиск часто прекращает проверку после первого или второго числа образца, поэтому его практическое время близко к линейному. Из-за этого КМП не получает заметного преимущества: на части запусков он немного быстрее, на части медленнее из-за дополнительных обращений к префикс-функции. В худшем случае разница становится существенной. При $|T| = 5\,000\,000$ наивный алгоритм тратит **55 733 мкс**, а КМП — **11 924 мкс**, то есть КМП быстрее примерно в **4.7 раза**. Это соответствует теоретической разнице между $O(|P| \cdot |T|)$ и $O(|P| + |T|)$.

5 Дневник выполнения работы

- 11.05.2026: изучено задание на поиск одного образца в тексте над алфавитом чисел от 0 до $2^{32} - 1$.
- 11.05.2026: реализовано построение префикс-функции и потоковый алгоритм Кнута—Морриса—Пратта без хранения всего текста в памяти.
- 11.05.2026: подготовлены основной генератор, генератор худшего случая и benchmark для сравнения с наивным поиском.
- 11.05.2026: проверены пересечения вхождений, переходы через границы строк, ведущие нули в числах и соответствие вывода формату контеста.

6 Выводы

Выполнив данную лабораторную работу по курсу «Дискретный анализ», я реализовал алгоритм Кнута—Морриса—Пратта для поиска образца в тексте над алфавитом целых чисел. Разобрался с построением префикс-функции и механизмом откатов, позволяющим избежать повторных сравнений. Реализовал корректную обработку ведущих нулей через стандартное считывание `uint32_t`, а также потоковый поиск вхождений, пересекающих границы строк, без хранения всего текста в памяти.

Тестирование подтвердило теоретические оценки: на случайных данных преимущество КМП не всегда заметно из-за коротких частичных совпадений, однако на патологических входах (образец из 19 единиц и завершающей двойки в тексте из единиц) алгоритм работает примерно в $4.7\times$ быстрее наивного поиска при $n = 5\,000\,000$, демонстрируя линейное время против квадратичного.

Список литературы

- [1] Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. *Алгоритмы: построение и анализ, 2-е издание*. — Издательский дом «Вильямс», 2007. — 1296 с. (ISBN 5-8459-0857-4 (рус.))
- [2] Knuth, D. E., Morris, J. H., Pratt, V. R. *Fast Pattern Matching in Strings*. SIAM Journal on Computing, 6(2):323–350, 1977.
- [3] Седжвик, Р. *Алгоритмы на C++. Часть 4: Поиск*. — Вильямс, 2004.